Deploying the Restaurant Kiosk App

A free, one-hour plan to host it next to your existing website on Oracle Cloud · 22 Oct 2026

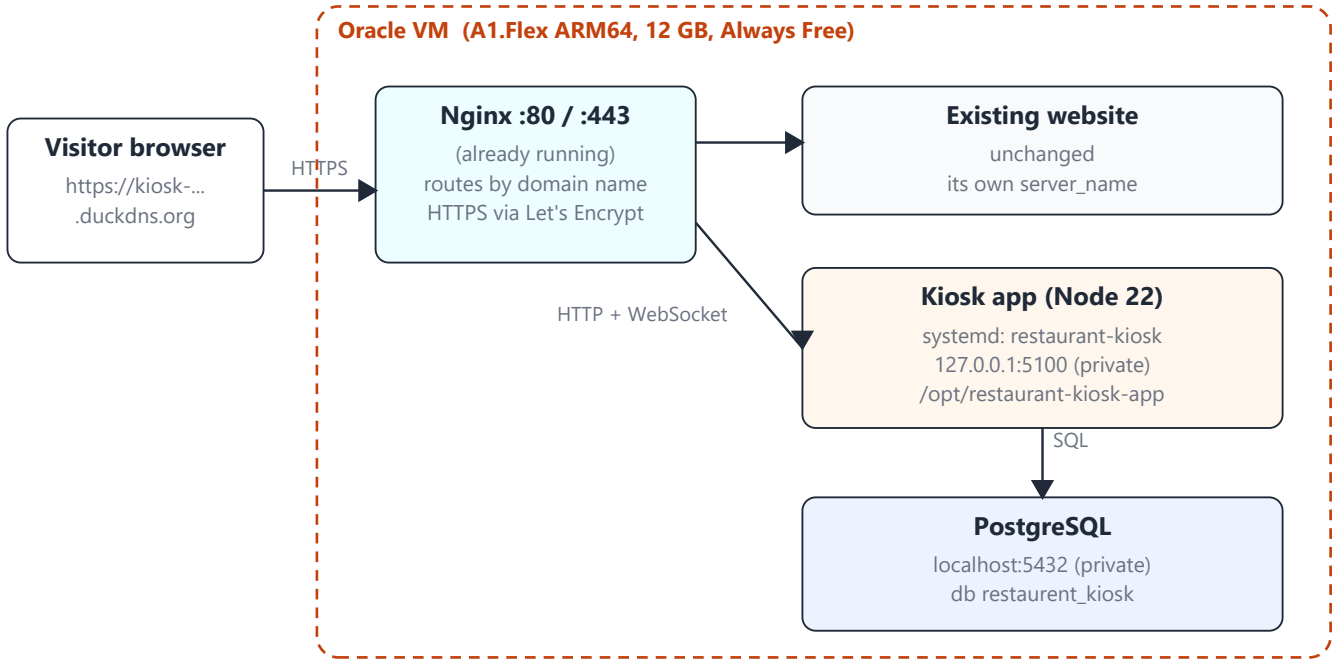
The short answer. Yes, both websites can run on your VM at the same time. A 12 GB ARM VM has far more capacity than two demo sites need. Together the kiosk app and its database use about 300 MB of RAM. Total extra cost: **Rs. 0**.

1. Why both sites fit on one VM

The limit isn't capacity, it's **collisions**: two apps fighting over the same port, Node version or web-server config. The plan keeps the kiosk app in its own lane at every layer:

Layer	What could clash	How the plan avoids it
Network ports	Both apps want 80/443	Nginx already owns 80/443 and routes each request **by domain name**. The kiosk app listens on a private port (5100) that the internet can't reach.
Node.js	Existing site may need another Node version	Node 22 is unpacked into **/opt/node-22** and only the kiosk service uses it. Nothing global changes.
Nginx config	Editing the live config could break the existing site	We **add one new file** and leave the old ones alone. Backup first, test with `nginx -t`, then reload (no downtime).
Database	Existing PostgreSQL data	Separate database and user. PostgreSQL listens on localhost only.
HTTPS	Certificates of the existing site	certbot is told only about the kiosk domain.
Firewall	Opening extra ports	Nothing new is opened. 80/443 are already open.

2. How a request reaches the kiosk app



1) The browser looks up your DuckDNS name and gets the VM's public IP. 2) Nginx receives the request, checks the domain name and forwards it to the kiosk app on 127.0.0.1:5100. 3) The app reads and writes orders in PostgreSQL. 4) The kitchen dashboard keeps a WebSocket open through Nginx, so new orders appear instantly. That's why the Nginx block includes the `Upgrade` headers.

3. What makes it completely free

Piece	Free because
VM	Already exists. Always Free covers 4 OCPU / 24 GB of A1 capacity.
OCI Bastion	No-charge OCI service, used once to recover SSH access, then deleted.
Domain name	DuckDNS gives free subdomains (e.g. rishi-kiosk.duckdns.org).
HTTPS certificate	Let's Encrypt via certbot, auto-renewed every 60-90 days.
Database	PostgreSQL runs on the same VM, so no paid managed database.
Payments	Razorpay Test Mode. With no keys at all, the app simulates payments.
Bandwidth	10 TB/month outbound free; a demo uses a tiny fraction of that.

Why HTTPS isn't optional here: with NODE_ENV=production the app marks its login cookie Secure, and browsers only send Secure cookies over HTTPS. Without a domain and certificate, login would silently fail. That's why the plan includes DuckDNS and certbot.

4. The one-hour timeline

#	Step	Who	Time
0	Commit local code and merge the GitHub README commits (done)	Claude	-
0b	**Push to GitHub:** git push origin main	**You**	1 min
1	Install OCI CLI on the laptop (done, v3.95.1)	Claude	-
2	**Add the API key in the OCI Console** and send the config preview	**You**	3 min
3	Configure CLI; find the VM's OCID, OS, public IP and subnet	Claude	3 min
4	Recover SSH: enable Bastion plugin, create Bastion, managed SSH session, add new key	Claude	10-15 min
5	**Create DuckDNS subdomain** pointing to the VM IP	**You**	3 min
6	Read-only inspection of the VM (Nginx layout, ports, Node, PostgreSQL)	Claude	3 min
7	Install Node 22 (isolated), PostgreSQL and certbot if missing	Claude	5 min
8	Create kiosk user, clone repo, write .env, npm ci, seed + migrate	Claude	5 min
9	systemd service and health check	Claude	2 min
10	Nginx site file, nginx -t, reload	Claude	3 min
11	HTTPS certificate, verify both websites	Claude	3 min
12	Delete Bastion, finish command log	Claude	2 min
13	Optional: Razorpay test keys and webhook, change demo passwords	You	5 min

5. Recovering SSH without the old key

Normally you'd need the private key you downloaded when the VM was created. Since it's gone, we use **OCI Bastion managed SSH**. The Oracle Cloud Agent already on your VM briefly accepts a key that we register through the OCI API. Over that temporary session we add a new permanent key (`~/.ssh/oci_vm_ed25519`). No reboot, so the existing website stays up.

If Bastion can't be used, the fallbacks are: (a) check OCI Cloud Shell's `~/.ssh` folder in case you connected from there before, or (b) the serial console in maintenance mode. Option (b) reboots the VM, so expect about 5 minutes of downtime.

6. Manual steps for you

M1. Push the code

The commit is ready locally, but pushing to GitHub needs your permission. In the project folder, run:

```
git push origin main
```

M2. Let the OCI CLI act for you (API key)

1) OCI Console, profile icon (top right), **My profile**, **API keys**, **Add API key**. 2) Choose **Paste a public key** and paste the contents of `C:\Users\Rishi\.oci\oci_api_key_public.pem`. 3) Click **Add**, copy the **Configuration file preview** and send it to Claude. It contains only IDs, a fingerprint and the region, with no secrets.

M3. Free domain from DuckDNS

1) Open duckdns.org and sign in with GitHub or Google. 2) Add a subdomain such as **rishi-kiosk**. 3) Put the VM's public IP (Claude prints it in step 3) into **current ip** and click **update ip**. 4) Send Claude the full name. Optionally also send the token, which is used for an auto-update cron job. 5) Check it: `nslookup rishi-kiosk.duckdns.org` should return the VM IP.

M4. Rotate secrets that leaked on GitHub

> The first commit in your **public** GitHub repo contains the `.env` file. The new commit stops tracking it, but anyone can still read the old commit. Regenerate your **Razorpay** keys (and PhonePe keys, if real ones were in there). The server gets brand-new random values for SECRET_KEY and the database password, so those are covered.

M5. After go-live (optional)

Send Razorpay **test** keys if you want real Razorpay test checkout; otherwise payments are simulated. Add the webhook URL `https://<kiosk domain>/api/payments/webhook` in the Razorpay dashboard. Change the seeded demo passwords: admin@ / staff@ / kiosk@example.com all use password123.

7. What gets configured on the server

Everything the kiosk app owns lives in a few predictable places, so removing it later is easy:

Path / name	Purpose
/opt/restaurant-kiosk-app	Code cloned from GitHub, owned by the **kiosk** Linux user
/opt/restaurant-kiosk-app/.env	Production settings and secrets (readable only by kiosk)
/opt/node-22	Private Node.js 22 runtime for this app only
restaurant-kiosk.service	systemd unit: starts on boot, restarts on crash
Nginx: restaurant-kiosk(.conf)	The one new site file that routes the DuckDNS name to port 5100
PostgreSQL: restaurent_kiosk / kiosk_app	Dedicated database and user

Production .env

```
PORT=5100
NODE_ENV=production
SECRET_KEY=<random, generated on the VM>
BUSINESS_TIMEZONE=Asia/Kolkata
CORS_ORIGIN=
DB_HOST=localhost
DB_PORT=5432
DATABASE_NAME=restaurent_kiosk
USER_NAME=kiosk_app
PASSWORD=<random, generated on the VM>
RAZORPAY_KEY_ID=          # empty = simulated payments
RAZORPAY_KEY_SECRET=
RAZORPAY_WEBHOOK_SECRET=
```

Nginx site block (the important part)

```
server {
    listen 80;
    server_name kiosk-<name>.duckdns.org;
    location / {
        proxy_pass http://127.0.0.1:5100;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;   # live kitchen dashboard
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

certbot later adds the HTTPS (443) part to this same block automatically.

8. Day-two operations

Task	Command (on the VM)
Deploy new code after a git push	`sudo -u kiosk /opt/restaurant-kiosk-app/deploy/update.sh && sudo systemctl restart restaurant-kiosk`
See live logs	`journalctl -u restaurant-kiosk -f`
Health check	`curl -s http://127.0.0.1:5100/healthz`
Stop the kiosk site	`sudo systemctl disable --now restaurant-kiosk`
Undo Nginx change	Delete the kiosk site file, then `sudo nginx -t && sudo systemctl reload nginx`

9. Glossary

Term	Meaning
OCI CLI	Oracle's command-line tool for driving your cloud account from a terminal instead of clicking in the Console.
Bastion	An OCI-managed jump host that opens a time-limited SSH tunnel into your VM.
Reverse proxy	Nginx receiving public traffic and forwarding it to a private app port.
server_name	The Nginx setting that picks which site handles a request, based on the domain typed in the browser.
systemd	Linux's service manager. It keeps the app running and starts it at boot.
Migration / seed	Seed creates tables and loads the demo menu and users (first deploy only). Migrations evolve the schema safely.